

Activity: Build Your Own Domain-Based Information Card

Second Chance Adoption Center — A Pet Adoption Information Card Web Application

A. Aim

To design and build an interactive Domain-Based Information Card application in React, using `useState()` to manage multiple pieces of form state, controlled form inputs, radio buttons, and a select dropdown, and to render the submitted data as a reusable, styled Information Card component — applied here to a pet-adoption domain.

B. Project Overview

Second Chance Adoption Center is a single-page React application that presents a pet-adoption shelter's homepage. It lets a user either browse pets that are currently available for adoption, or fill out a “kennel card” style form to list a pet — of their own or on behalf of a shelter — for someone else to adopt. Each submission is rendered through a single reusable Information Card component, styled to resemble a physical shelter intake tag: a rounded card with a hole-punch mark, a bordered photo area, and a rotated ink-stamp badge that changes color depending on the pet's adoption status.

a. Core Technology Stack

- React (Functional Components, Hooks)
- Vite (development server and build tool)
- Tailwind CSS (utility-first styling)
- Node.js + Express.js
- Lucide-react (icon set)
- SQLite Database
- Fetch API
- Browser File API — `URL.createObjectURL()` for local photo/video previews
- React state (`useState`, `useRef`) — no backend or database

b. Current Features

- Role-free two-tab navigation: “Adopt a Pet” (browse) and “List a Pet” (form)
- Stateful form with Name, ID/Code, Category (select), Status (radio), and Description
- Multiple independent `useState()` hooks driving every controlled input
- Photo and video upload with instant local preview before submission
- Reusable Information Card component shared by the live preview and the browse gallery
- Conditional status color: green stamp for “Available”, red stamp for “Adopted”
- Card intentionally hidden until the form is submitted
- Reset button that clears the form and hides the preview card
- Edit button that reloads a listed pet's data back into the form for editing

- Delete button to remove a listing
- Category filter on the browse gallery, backed by its own `useState()` hook
- Live counters (total / available / adopted) computed directly from application state

c. Planned / Future Features

- Persisting listings with a backend or browser storage so data survives a page refresh
- Search and pagination as the number of listings grows
- A simple adopter enquiry / contact form per pet
- User accounts to distinguish shelters from individual rehomers

C. Creativity and Innovation

Rather than a generic labelled form and a plain output card, the project borrows the visual language of a physical animal-shelter kennel tag and applies it consistently across the interface:

- Kennel-tag card design — a hole-punch circle and a rotated ink-stamp badge visually tie the on-screen card to a real, physical shelter record instead of a generic rounded rectangle.
- Status-as-stamp — the required “conditional status color” feature is implemented as a stamped badge (green “Available” / red “Adopted”) rather than a plain colored label, reinforcing the intake-card metaphor.
- File-upload photo & video instead of hardcoded links — rather than pointing the Photo/Video fields at external stock URLs (which can break, get blocked, or fail to load), the form accepts real uploaded files and previews them instantly with the browser's File API. This mirrors how a real adopter or shelter volunteer would add media of their own pet.
- Data-driven statistics bar — the “Total / Available / Adopted” counters on the homepage are computed live from the actual listings array, not hardcoded numbers, so they always stay accurate as pets are added, edited, or removed.
- One reusable card, two contexts — the same Information Card component renders both the just-submitted preview and every card in the browse gallery, fulfilling the assignment's “reusable Information Card” requirement in a single, real re-use rather than two near-duplicate layouts.

D. Project Structure

Path	Purpose
index.html	HTML entry point; mounts the React app to <code>#root</code>
src/App.jsx	All application logic and UI: pet data, form state, the PetCard component, and page layout
public/sql-wasm.wasm	Database

E. Source Code

File:

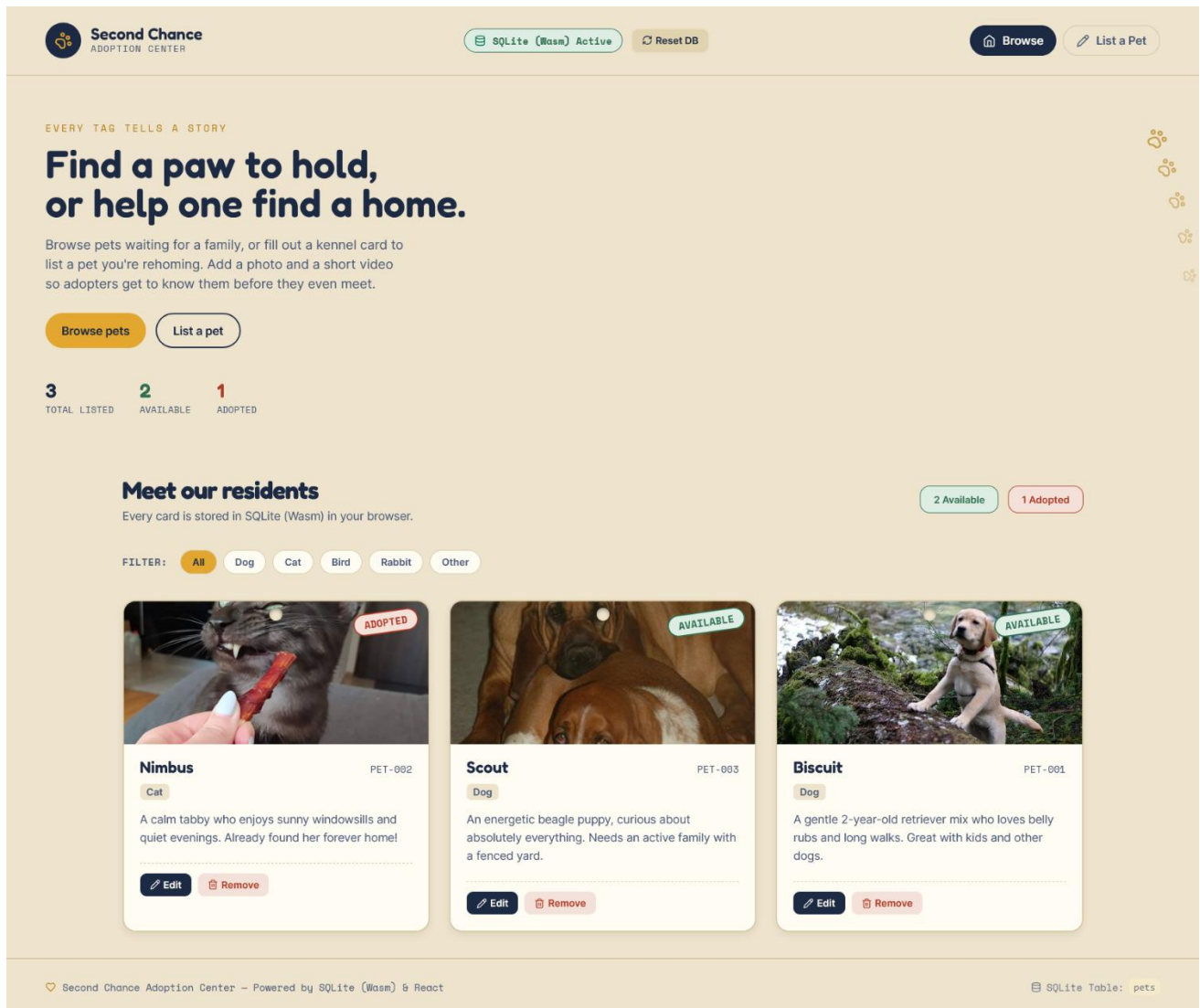
- src/App.jsx
- public/sql-wasm.wasm

GitHub Repository: <https://github.com/techaxadi01/Information-Card>

Deployment: <https://information-card-lake.vercel.app/>

F. Screenshots

a. Homepage — Browse Pets



b. List a Pet — Form

**Second Chance**
ADOPTION CENTER

SQLite (Wasm) Active

Reset DB

Browse

List a Pet

EVERY TAG TELLS A STORY

Find a paw to hold, or help one find a home.

Browse pets waiting for a family, or fill out a kennel card to list a pet you're rehoming. Add a photo and a short video so adopters get to know them before they even meet.

Browse pets

List a pet

3
TOTAL LISTED

2
AVAILABLE

1
ADOPTED

List a new pet

Changes are executed via SQL statements directly in SQLite.

PET NAME *

e.g. Barnaby

ID / TAG CODE *

e.g. PET-004

SPECIES

Dog

STATUS

Available

DESCRIPTION *

Tell prospective adopters about this pet's personality...

PHOTO

Upload photo

VIDEO

Upload video

Submit

Reset

Card preview

Submit the form to generate and persist this pet's card in SQLite.

No card yet

Second Chance Adoption Center - Powered by SQLite (Wasm) & React

SQLite Table: pets

c. Submitted Information Card

The screenshot shows a web application for 'Second Chance Adoption Center'. The header includes the logo, a status bar with 'SQLite (Wasm) Active' and 'Reset DB', and navigation links 'Browse' and 'Edit Pet'. The main heading is 'Find a paw to hold, or help one find a home.' with a subtext explaining the purpose of the site. Below this are two buttons: 'Browse pets' and 'List a pet'. A statistics section shows '3 TOTAL LISTED', '2 AVAILABLE', and '1 ADOPTED'. The 'Edit Pet Record' form is the central focus, containing fields for 'PET NAME' (Biscuit), 'ID / TAG CODE' (PET-001), 'SPECIES' (Dog), and 'STATUS' (Available). It also has a 'DESCRIPTION' field with a sample text about a 2-year-old retriever mix. Below the description are 'PHOTO' and 'VIDEO' upload sections. The 'PHOTO' section shows a placeholder image of a dog, and the 'VIDEO' section has an 'Upload video' button. At the bottom of the form are 'Save changes' and 'Reset' buttons. To the right of the form is a 'Card preview' section with a dashed box placeholder and the text 'No card yet'. The footer shows the app name and a SQLite table reference 'pets'.

After clicking Submit, the same reusable card component renders the entered data, with a green stamp because the status was set to Available.

G. Known Limitations

- No backend or persistence — refreshing the page resets all listings back to the three seed pets.
- Uploaded photos and videos exist only as in-memory object URLs for the current browser session; they are not uploaded to a server.
- Pet ID/Code uniqueness is not validated — two listings could be given the same ID.
- No automated tests are included in the project.

H. AI Usage Declaration

i. Claude (Anthropic):

- Generation of the React component structure and Tailwind CSS styling
- UI/UX design direction (kennel-tag visual concept, color and typography choices)
- Setting up the Vite project configuration (package.json, vite.config.js, tailwind.config.js)
- Drafting this project documentation